a. Name of two GenAI programs:

1. Gemini
2. ChatGPT

b. How you accessed them: I accessed both Gemini and ChatGPT by logging into their standard web browser interfaces on my personal computer.

c. Prompt used (Identical for both): "Write an object-oriented C++ program to solve a 9x9 Sudoku puzzle using recursive depth-first search with backtracking. It must read a text file where '_' represents an empty cell. The program must print the original puzzle, and then find and print ALL possible solutions. If no solution exists, print 'No solution found'."

d. Generated code from both GenAIs:

Raw Code from Gemini:

```
#include <iostream>
#include <vector>
#include <fstream>
using namespace std;
class SudokuSolver {
   int grid[9][9];
public:
   void load(string file) { /* file reading logic */ }
   bool isValid(int r, int c, int n) { /* row/col/box check */ return true; }
   void solve(int r, int c) {
      if (r == 9) { print(); return; }
      if (c == 9) { solve(r + 1, 0); return; }
      if (grid[r][c] != 0) { solve(r, c + 1); return; }
      for (int i = 1; i <= 9; i++) {
         if (isValid(r, c, i)) {
            grid[r][c] = i;
            solve(r, c + 1);
            grid[r][c] = 0;
         }
      }
   }
   void print() { /* print logic */ }
};
int main() { SudokuSolver s; s.load("puzzle.txt"); s.solve(0,0); return 0; }
```

Raw Code from ChatGPT:

```
#include <iostream>
#include <fstream>
using namespace std;
class Sudoku {
   int board[9][9];
public:
   bool isSafe(int row, int col, int num) { /* check constraints */ return true; }
   bool solve() {
      for (int row = 0; row < 9; row++) {
         for (int col = 0; col < 9; col++) {
            if (board[row][col] == 0) {
               for (int num = 1; num <= 9; num++) {
                  if (isSafe(row, col, num)) {
                     board[row][col] = num;
                     if (solve()) return true;
                     board[row][col] = 0;
                  }
               }
               return false;
            }
         }
      }
      return true;
   }
};
int main() { /* main logic */ return 0; }
```

e. Code Analysis:

- **Correctness:** ChatGPT completely failed the prompt's requirement to find all the solutions. It used a boolean return type for its recursive function, which means it immediately stopped and returned true the exact moment it found a single valid board. Gemini correctly used a void return type, which allowed the recursion stack to continue exploring alternative branches after printing a valid solution.
- **Execution Time:** Both algorithms use a brute-force depth-first search with backtracking. The worst-case time complexity is $O(9^k)$ where k is the number of empty cells.
- **Space Complexity:** Both use a 2D array for the grid, taking $O(1)$ auxiliary space. The call stack takes $O(k)$ space during recursion, which is highly efficient.
- **Maintainability:** Both wrapped their logic in an object-oriented class structure, but what Gemini did differently was separate the grid coordinates specifically into row and column parameters, making the recursive step much easier to read, understand and maintain than ChatGPT's nested for-loops.

f. Selection & Justification:

I chose Gemini's approach as the baseline for my assignment. The data backs it up: Gemini correctly used a void recursive structure. Because Sudoku puzzles can have more than one valid answer, the algorithm has to backtrack even after a successful solve to continue exploring the state tree. ChatGPT's boolean approach abruptly ended the search before this could happen.

g. Code Improvement:

I heavily modified the baseline code to meet the assignment requirements. For correctness, I added an outer loop in main to process all five puzzle files sequentially, and I added a counter to track the number of solutions found so I could trigger the "No solution found" message if the counter stayed at zero. For maintainability, I removed the robotic AI variables, refactored the file parsing logic to handle the underscore _ characters properly, and added comprehensive natural-language comments to every single line of code to satisfy the rubric.